AI/ML Job-Ready Roadmap & Interview Answer Guide

20 topics. Structured answers. Real engineering scenarios.

Prepare for ML Engineer and GenAI Engineer conversations with answers you can explain, adapt and defend. This guide combines a practical study plan with interview practice.

INSIDE THE GUIDE

A reusable answer framework • 20 core interview questions • Beginner explanations and technical answers • Python/SQL examples • Follow-up prompts • 5 production scenarios • A four-week roadmap

HOW TO USE THIS GUIDE

1. Read the beginner answer to understand the idea.
2. Close the page and answer aloud in 60-90 seconds.
3. Add an example from a project you actually built.
4. Explain one tradeoff and how you would test your decision.



Prepared by

learnwithbhawana

This is a study resource, not a job guarantee. Technical references checked 3 November 2026.

How to answer with structure

USE THIS FIVE-STEP ANSWER

1. Clarify: What is the target, data, constraint and success metric?
2. Explain: State the core idea in plain language.
3. Apply: Describe a concrete approach and why it fits.
4. Evaluate: Give a metric, baseline and validation plan.
5. Discuss tradeoffs: Name a limitation, failure case or alternative.

EXAMPLE OPENING

“Before choosing a model, I’d clarify the prediction time, available features and cost of errors. I’d start with a simple baseline, validate on a split matching production, and compare improvements against latency and maintenance costs.”

FOR CODING ROUNDS

Clarify inputs and edge cases. Explain a simple solution first. Improve time or memory if needed. Write readable code, then test empty input, duplicates, nulls and boundary values. State complexity.

FOR PROJECT QUESTIONS

Use: problem → your contribution → baseline → data/split → approach → measured result → limitation. Only quote numbers you measured. Explain your own decisions, including one failed experiment.

WHEN YOU DO NOT KNOW

Say what you know, state your assumption, reason from fundamentals and explain how you would verify it. Example: “I haven’t used that library, but I’d compare it using the same held-out tasks, latency budget and failure cases.”

STUDY MAP / PAGES

03 Python + SQL • 04 Statistics + Pandas • 05 Preprocessing + Features
06 Algorithms + Bias/Variance • 07 Evaluation + Imbalance
08 Tuning + Deep Learning • 09 Transformers + RAG
10 Fine-tuning + Agents • 11 Deployment + MLOps
12 Monitoring + System Design • 13-14 Scenarios
15 Roadmap • 16 Mock interview checklist + references

Code snippets are teaching examples. Unless marked standalone, they assume the named data and imports exist; adapt them to your dataset and installed library version.

01-02 / Interview practice

01 / Python Coding

INTERVIEWER ASKS

How would you process a large file without loading everything into memory?

BEGINNER ANSWER

Use a generator to produce one item at a time. It avoids building a full list and supports streaming.

INTERVIEW-READY ANSWER

I would iterate over the file and yield parsed rows. A generator is lazy and preserves execution state between yields. I would handle malformed rows explicitly, close resources with a context manager and measure memory. Streaming reduces stored data, but downstream list conversion removes that benefit.

```
def rows(path):
    with open(path) as f:
        for line in f:
            yield line.strip()
```

FOLLOW-UP + ANSWER CUE

Generator vs list? A list stores all items; a generator is usually single-pass and computes on demand.

Avoid: Claiming generators always run faster. Also practice dictionaries, OOP, decorators and mutable defaults.

02 / SQL

INTERVIEWER ASKS

How do you find the latest event for each user?

BEGINNER ANSWER

Assign a row number within each user, ordered by event time. Keep row number one.

INTERVIEW-READY ANSWER

I would use ROW_NUMBER with PARTITION BY user_id and a descending timestamp. I would add a unique event ID as a tie-breaker and define how null timestamps behave. Then I would inspect the query plan and indexes/partitioning on representative data. Joins can multiply rows, so I would verify the expected output grain.

```
WITH ranked AS (
 SELECT *, ROW_NUMBER() OVER (PARTITION BY user_id
 ORDER BY event_time DESC, event_id DESC) AS rn
 FROM events WHERE event_time IS NOT NULL
) SELECT * FROM ranked WHERE rn = 1;
```

FOLLOW-UP + ANSWER CUE

ROW_NUMBER vs RANK? RANK preserves ties and may return multiple first-place rows.

Avoid: Assuming a CTE automatically improves performance; behavior depends on the engine.

03-04 / Interview practice

03 / Statistics

INTERVIEWER ASKS

What does a p-value of 0.03 mean?

BEGINNER ANSWER

Assuming the null hypothesis and test assumptions hold, results at least this extreme have probability 3%. It is not a 97% chance your hypothesis is true.

INTERVIEW-READY ANSWER

I would first define the hypothesis, metric and sampling unit. A p-value measures incompatibility with a specified null model, not effect size or business importance. I would report the estimated effect and confidence interval, check assumptions and account for multiple comparisons. Randomization and adequate power matter for interpreting an experiment.

```
Example: a small conversion lift can be statistically significant
with a huge sample but still fail the business ROI threshold.
```

FOLLOW-UP + ANSWER CUE

What is statistical power? The chance of rejecting a specified false null under a chosen effect size and design.

Avoid: Interpreting “not significant” as proof of no effect.

04 / NumPy & Pandas

INTERVIEWER ASKS

Why prefer vectorized operations over row-wise apply?

BEGINNER ANSWER

Vectorized operations often use optimized array code and avoid repeated Python function calls.

INTERVIEW-READY ANSWER

I would use column operations, boolean masks and groupby aggregations before row-wise apply. I would check dtypes, missing values and index alignment because fast code can still be incorrect. I would benchmark on representative data and track memory: vectorization can create large intermediate arrays.

```
import pandas as pd
df = pd.DataFrame({"team": ["A", "A", "B"],
                   "sales": [10, 20, 5]})
totals = df.groupby("team", as_index=False)["sales"].sum()
```

FOLLOW-UP + ANSWER CUE

What can go wrong with chained assignment? Updates can be ambiguous; use explicit .loc assignments.

Avoid: Assuming np.vectorize provides the same speed benefit as a compiled NumPy operation.

05-06 / Interview practice

05 / Data Preprocessing

INTERVIEWER ASKS

How do you prevent data leakage while filling missing values?

BEGINNER ANSWER

Split first. Learn imputation values from training data only and reuse them for validation and test data.

INTERVIEW-READY ANSWER

I would match the split to deployment, using time or grouped splits when appropriate. I would put imputation, encoding and the estimator in a pipeline so each CV fold learns preprocessing only from its training portion. I would audit whether features exist at prediction time and keep the final test set untouched.

```
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
model = make_pipeline(SimpleImputer(strategy="median"),
                      LogisticRegression(max_iter=1000))
```

FOLLOW-UP + ANSWER CUE

Is a high score proof of leakage? No; investigate feature timing, duplicate entities and split construction.

Avoid: Fitting encoders, scalers or outlier thresholds on the entire dataset before splitting. [1]

06 / Feature Engineering

INTERVIEWER ASKS

How do you decide whether a new feature is useful?

BEGINNER ANSWER

Compare the same model with and without the feature on a realistic validation split. Check that the feature is available at inference time.

INTERVIEW-READY ANSWER

I would begin with a domain hypothesis and prediction-time availability. I would test the feature through cross-validation, compare relevant metrics and check performance across segments. I would consider stability, redundancy, computation cost and leakage. Feature selection must happen inside training folds; final decisions should not repeatedly use the test set.

```
Example: past_30d_orders uses only events before prediction time.
A future cancellation flag would leak the outcome.
```

FOLLOW-UP + ANSWER CUE

When is scaling important? For distance-based and many gradient/regularized models; trees usually need it less.

Avoid: Treating feature importance as causal evidence or comparing different splits.

07-08 / Interview practice

07 / ML Algorithms

INTERVIEWER ASKS

Would you choose logistic regression, random forest or XGBoost?

BEGINNER ANSWER

Start with a simple baseline. More complex models may capture nonlinear patterns, but they add tuning and serving costs.

INTERVIEW-READY ANSWER

I would clarify data size, feature types, interpretability and latency. Logistic regression is a useful linear classification baseline; random forests average randomized trees; gradient boosting adds trees sequentially to improve an objective. I would compare them using the same leakage-safe validation and business metric, then choose the simplest model meeting requirements.

```
Example: compare PR performance, calibration and p95 latency
for a rare-event classifier, not just one accuracy score.
```

FOLLOW-UP + ANSWER CUE

Why can boosting overfit? Excess capacity or too many iterations can fit noise; use regularization and validation.

Avoid: Saying XGBoost is always best or that logistic regression is a regression model for continuous targets.

08 / Bias vs Variance

INTERVIEWER ASKS

Training performance is strong, but validation performance is poor. What next?

BEGINNER ANSWER

The model may be overfitting, but first check that training and validation represent the same task and data conditions.

INTERVIEW-READY ANSWER

I would inspect learning curves, split quality and possible leakage or distribution differences. If overfitting is supported, I would reduce complexity, increase regularization, use early stopping or add useful data. If both training and validation are weak, I would investigate underfitting, labels and feature quality. I would compare changes on a stable validation protocol.

```
Example: tune tree max_depth and min_samples_leaf;
compare training and validation scores across folds.
```

FOLLOW-UP + ANSWER CUE

Can more data fix high bias? Often not alone; better features or a more expressive model may be needed.

Avoid: Diagnosing bias or variance from one score without checking data and evaluation.

09-10 / Interview practice

09 / Model Evaluation

INTERVIEWER ASKS

Your classifier has 98% accuracy. Is it good?

BEGINNER ANSWER

Not necessarily. If 98% of cases are negative, always predicting negative achieves 98% accuracy and detects no positives.

INTERVIEW-READY ANSWER

I would inspect prevalence and the confusion matrix, then select metrics from error costs. Precision measures how many positive predictions are correct; recall measures how many positives are found. I would inspect the precision-recall curve, calibration and useful operating thresholds, tune on validation data and report final test results. ROC-AUC alone can hide poor rare-class precision.

```
TP, FP, FN = 40, 10, 60
precision = TP / (TP + FP)  # 0.80
recall = TP / (TP + FN)     # 0.40
```

FOLLOW-UP + ANSWER CUE

When prioritize precision? When false positives are costly; explain the recall tradeoff and actual business cost.

Avoid: Using a default 0.5 threshold without justification. Average precision and trapezoidal PR-AUC are not identical.

10 / Imbalanced Datasets

INTERVIEWER ASKS

How would you build a fraud classifier with 1% positive cases?

BEGINNER ANSWER

Use a realistic split and focus on positive-class performance. Class weights, resampling and threshold tuning are options to test.

INTERVIEW-READY ANSWER

I would establish a baseline and evaluate precision/recall at an operational review capacity. I would compare class weights with resampling, applying SMOTE only within training folds. Synthetic interpolation needs suitable features; categorical data and time-dependent fraud require care. I would tune the threshold on validation data and assess calibration and delayed labels.

```
Example: choose a threshold that keeps alerts within the
review team capacity, then measure recall at that workload.
```

FOLLOW-UP + ANSWER CUE

Why not resample the test set? It should reflect the deployment population and natural prevalence.

Avoid: Applying SMOTE before splitting or assuming balanced training data gives calibrated probabilities.

11-12 / Interview practice

11 / Hyperparameter Tuning

INTERVIEWER ASKS

How do you tune a model without overfitting validation data?

BEGINNER ANSWER

Use cross-validation on the training data and reserve a final test set. Tune a limited search space based on model behavior.

INTERVIEW-READY ANSWER

I would select a deployment-appropriate CV strategy, pipeline and scoring metric. Grid search suits a small discrete space; randomized search covers larger spaces under a budget; Optuna can guide sequential trials and pruning. I would track seeds, data versions and trial budgets. For an unbiased estimate after extensive selection, consider nested CV or a separate untouched test set.

```
from sklearn.model_selection import RandomizedSearchCV
# estimator and cv reflect your data and split strategy.
search = RandomizedSearchCV(estimator, param_distributions,
    n_iter=20, scoring="average_precision", cv=cv, random_state=42)
```

FOLLOW-UP + ANSWER CUE

What is a hyperparameter? A setting chosen outside parameter fitting, such as depth or learning rate.

Avoid: Repeatedly checking the test set while selecting the best configuration.

12 / Deep Learning

INTERVIEWER ASKS

Explain backpropagation and diagnose a network that will not learn.

BEGINNER ANSWER

Backpropagation uses the chain rule to compute loss gradients. An optimizer uses those gradients to update weights.

INTERVIEW-READY ANSWER

I would verify labels, input scales and the loss/output pairing, then try to overfit a tiny batch. I would inspect gradient norms, learning rate, initialization and training/evaluation modes. SGD and Adam update parameters differently, but neither fixes a wrong objective. I would monitor validation loss and apply regularization or early stopping when generalization degrades.

```
# Framework-neutral training loop:
# clear gradients -> forward -> compute loss
# backward -> optimizer step -> log metrics
```

FOLLOW-UP + ANSWER CUE

Why exploding/vanishing gradients? Repeated transformations can amplify or shrink gradients; discuss initialization and architecture.

Avoid: Using softmax twice with a loss function that already expects raw logits.

13-14 / Interview practice

13 / Transformers & LLMs

INTERVIEWER ASKS

What does self-attention do?

BEGINNER ANSWER

It lets each token combine information from other permitted tokens, weighted by relevance. Tokens are subword units, not always whole words.

INTERVIEW-READY ANSWER

Queries and keys produce similarity scores; scaling, masking and softmax produce attention weights over values. Multiple heads learn different relationships. Positional information represents order, while causal masking blocks future tokens in autoregressive models. Embeddings are learned vector representations. Standard full attention has quadratic sequence-length cost in its attention matrix.

```
Attention(Q,K,V) = softmax(Q K^T / sqrt(d_k) + mask) V
# mask disallows positions, e.g. future tokens in a decoder.
```

FOLLOW-UP + ANSWER CUE

Why a KV cache? It reuses past keys/values during decoding, saving recomputation at a memory cost.

Avoid: Saying attention weights are a complete explanation of the model's reasoning.

14 / RAG Architecture

INTERVIEWER ASKS

How would you build and evaluate a document Q&A; assistant?

BEGINNER ANSWER

Retrieve relevant document passages, give them to the LLM and ask for an answer grounded in those passages with citations.

INTERVIEW-READY ANSWER

I would ingest and version documents, chunk them with metadata and access controls, then index embeddings and possibly lexical terms. At query time I would retrieve, optionally rerank, and build a bounded prompt with sources. I would evaluate retrieval recall separately from answer correctness, citation support and abstention. I would test permission filtering, prompt injection, stale documents, latency and cost.

```
ingest -> chunk -> embed/index
query -> retrieve -> permission filter -> rerank
      -> grounded generation -> answer + source citations
```

FOLLOW-UP + ANSWER CUE

How choose chunk size? Tune on representative questions; balance context completeness, retrieval precision and token cost.

Avoid: Assuming a vector database prevents hallucinations or guarantees authorization.

15-16 / Interview practice

15 / Fine-Tuning LLMs

INTERVIEWER ASKS

When would you fine-tune instead of using RAG?

BEGINNER ANSWER

RAG supplies external knowledge. Fine-tuning can improve task behavior, style or output format. They can be combined.

INTERVIEW-READY ANSWER

I would first test prompting and retrieval baselines. For a stable task with high-quality examples, I would fine-tune and compare on held-out tasks, including regressions and safety cases. LoRA trains low-rank adapters with base weights frozen; QLoRA combines a quantized frozen base with trainable adapters to reduce memory. PEFT is the broader family. I would track licensing, data provenance and serving costs.

```
Decision example: current internal policies -> retrieval.
Consistent specialist response format -> consider fine-tuning
only after prompt/schema baselines fail.
```

FOLLOW-UP + ANSWER CUE

Does QLoRA train every base weight in 4-bit? No; the base is typically frozen and adapters are trained.

Avoid: Treating fine-tuning as a reliable way to update a frequently changing knowledge base. [2]

16 / AI Agents

INTERVIEWER ASKS

When would you use an agent instead of a fixed workflow?

BEGINNER ANSWER

Use a workflow when steps are predictable. An agent can choose tools or next steps when the path varies, but needs limits and evaluation.

INTERVIEW-READY ANSWER

I would define the task, allowed tools and success criteria first. I would prefer deterministic orchestration where possible, then add model-driven routing where it improves outcomes. Tools need typed inputs, validation, scoped permissions, timeouts and retry limits. I would log traces, bound steps/cost, separate untrusted content from instructions and require approval for consequential actions. LangGraph is one orchestration option.

```
request -> plan/route -> validated tool call -> observation
        -> check completion or stop at step/cost limit
```

FOLLOW-UP + ANSWER CUE

What is memory? Distinguish short-lived task state from stored long-term information with retention and access rules.

Avoid: Giving an agent unrestricted tools or treating a successful demo as reliability evidence.

17-18 / Interview practice

17 / Model Deployment

INTERVIEWER ASKS

How would you serve a model with FastAPI and Docker?

BEGINNER ANSWER

Package the model and preprocessing together. Expose a validated endpoint, containerize dependencies and test the service under realistic load.

INTERVIEW-READY ANSWER

I would load a versioned artifact at startup, validate request schemas and apply the exact training transformations. I would add health/readiness checks, authentication, timeouts and structured logs without sensitive inputs. I would test p95 latency, throughput, memory and prediction parity before a canary rollout, with rollback ready. Batch inference may be cheaper when real-time responses are unnecessary.

```
# Endpoint contract example, not a full service:
# POST /predict {"features": [...]}
# -> {"score": 0.73, "model_version": "v3"}
```

FOLLOW-UP + ANSWER CUE

Why not load the model on every request? It adds latency and resource churn; load per worker with capacity planning.

Avoid: Assuming a Docker image alone gives scalability, security or reproducibility.

18 / MLOps & CI/CD

INTERVIEWER ASKS

What should an ML CI/CD pipeline validate?

BEGINNER ANSWER

Test code, data and model behavior, then promote a versioned artifact only when agreed quality and operational checks pass.

INTERVIEW-READY ANSWER

I would version code, environment, data references and features; log runs, parameters, metrics and artifacts in a tracker such as MLflow. CI would check schemas, transformations and prediction contracts. A training pipeline would compare candidates against the deployed baseline on held-out data and key segments. CD would use a registry, approval rules, staged rollout, monitoring and rollback. Retraining triggers should not imply automatic promotion.

```
commit -> tests -> data checks -> train/evaluate
       -> register candidate -> canary -> monitor/promote
```

FOLLOW-UP + ANSWER CUE

CI vs continuous training? CI integrates tested code; continuous training refreshes model candidates as data changes.

Avoid: Versioning only model weights while losing data, preprocessing or environment lineage. [3]

19-20 / Interview practice

19 / Model Monitoring

INTERVIEWER ASKS

How do you know a deployed model is getting worse?

BEGINNER ANSWER

Monitor inputs, service health and outcomes. Changing data is a warning, but it does not by itself prove prediction quality declined.

INTERVIEW-READY ANSWER

I would track schema issues, missingness and feature distributions alongside latency, errors and cost. With labels, I would measure task metrics and calibration by segment, accounting for label delay and selection bias. Data drift concerns input distributions; concept drift concerns the relationship between inputs and outcomes. I would investigate alerts, compare cohorts, and use rollback or retraining only after diagnosis.

```
Example dashboard: missing rate, p95 latency, alert volume,
precision on matured labels, and segment-level recall.
```

FOLLOW-UP + ANSWER CUE

What if labels arrive after 30 days? Use early data/service checks, then backfill outcome metrics on mature cohorts.

Avoid: Automatically retraining on every drift alert or calling drift the same thing as performance loss.

20 / ML System Design

INTERVIEWER ASKS

Design a recommendation system for a large application.

BEGINNER ANSWER

Clarify the product goal, scale and latency budget. Separate candidate retrieval from ranking, then measure both model and product outcomes.

INTERVIEW-READY ANSWER

I would define the objective, event schema and point-in-time training labels. A candidate stage narrows the catalog, a ranker scores items and a final stage applies constraints such as availability/diversity. I would address cold starts, feature freshness, caching, batch vs online computation and fallbacks. I would evaluate offline ranking metrics, then a guarded online experiment with latency, cost and user-impact checks.

```
events -> validated feature pipelines -> training/registry
request -> candidate retrieval -> rank -> constraints
        -> recommendations -> feedback + monitoring
```

FOLLOW-UP + ANSWER CUE

How handle a new user? Use contextual/popularity baselines, available onboarding signals and exploration within safeguards.

Avoid: Starting with a complex architecture before estimating traffic, freshness, latency and business needs.

Production questions

01 / GREAT OFFLINE SCORE, POOR PRODUCTION PERFORMANCE

Question: Your model works in training but fails after deployment. What do you check?
Answer: “I’d clarify whether failure means errors, latency or prediction quality. I’d compare training and serving schemas, preprocessing, model versions and feature timing. Next I’d examine split leakage, delayed labels and changes in user/data segments. I’d replay a known batch through both paths and isolate differences. If impact is material, I’d roll back or use a safe fallback while investigating.”
Evidence: Prediction parity, feature freshness and performance on mature production cohorts.
Follow-up: What if serving features are computed differently? Share transformations or enforce parity tests.

02 / RAG RETRIEVES PLAUSIBLE BUT IRRELEVANT DOCUMENTS

Question: Your assistant confidently gives the wrong policy. What changes?
Answer: “I’d inspect the trace and separate retrieval failure from generation failure. I’d verify document versions, permissions, chunk boundaries and query coverage. I’d test hybrid retrieval and reranking on labeled questions, then check whether the answer is supported by retrieved evidence. I’d add abstention for missing/contradictory evidence and validate citations.”
Evidence: Retrieval recall@k, supported-answer rate, unanswerable-query behavior and latency.
Follow-up: What if the correct document was retrieved? Review prompt construction, conflicting context and source selection.

03 / HIGH FRAUD RECALL OVERWHELMS REVIEWERS

Question: Recall increased, but the operations team cannot handle the alerts. What now?
Answer: “I’d quantify daily review capacity and the cost of missed fraud versus false alerts. I’d inspect precision-recall behavior, segment results and score calibration. I’d tune a validation threshold or top-k policy to match capacity, test routing for high-value cases and evaluate on an untouched, time-based cohort.”
Evidence: Precision and recall at review capacity, captured fraud value and total expected cost.
Follow-up: How do you avoid biased feedback? Track which cases are reviewed; consider safe auditing samples outside the alert queue.

Reliability under pressure

04 / ENDPOINT LATENCY DOUBLES AFTER A RELEASE

Question: The model looks accurate, but p95 latency doubled. How do you respond?
Answer: “I’d compare release versions and trace preprocessing, queueing, model execution and downstream calls. I’d check input lengths, batch sizes, cold starts, memory and concurrency. I’d reproduce at realistic traffic and compare a known-good version. I’d roll back if service objectives are breached, then evaluate caching, batching or a smaller model using both quality and latency tests.”
Evidence: Stage timings, p50/p95/p99 latency, throughput, errors and quality under load.
Follow-up: Does batching always help? It improves throughput in some workloads but can increase waiting time.

05 / AGENT REPEATS A TOOL CALL AND CREATES DUPLICATE ACTIONS

Question: A tool-using assistant retries a timed-out action and creates duplicates. How do you fix it?
Answer: “I’d stop further writes and inspect tool traces and operation IDs. A timeout does not prove failure, so I’d query action status before retrying. I’d add idempotency keys, durable state, bounded retries and tool-level validation. I’d gate consequential actions with explicit approval and test replay, partial failures and malicious tool outputs before release.”
Evidence: Duplicate-action rate, recovery success, approved-action correctness and step/cost budgets.
Follow-up: Is prompting alone enough? No; enforce these controls in the tool/service and orchestration layers.

SCENARIO ANSWER TEMPLATE

“I’d clarify impact and scope, stabilize the system, inspect evidence, rank hypotheses, run a small diagnostic test, and verify the fix against both quality and operational metrics.”

Always explain what evidence would change your diagnosis. Interviewers learn more from your debugging sequence than from a long list of tool names.

Four weeks of focused preparation

Suggested pace: 60-90 minutes per day, 6 days per week. Extend this plan if the concepts are new; readiness depends on demonstrable skills, not a calendar.

WEEK 1 / FOUNDATIONS + DATA

Study topics 01-06. Practice Python collections, generators, OOP and decorators; SQL joins, CTEs and windows; probability and hypothesis testing. Clean a small dataset with Pandas and build a leakage-safe split.
Deliverable: A notebook with a data dictionary, preprocessing pipeline and short explanation of split choices.
Checkpoint: Solve one Python and one SQL problem aloud without copying.

WEEK 2 / TRAIN + EVALUATE

Study topics 07-12. Build a simple baseline and one tree ensemble. Compare metrics on fixed splits, handle imbalance and tune within a budget. Explain gradients, regularization and learning curves.
Deliverable: An experiment table with baseline, candidate, segment metrics, threshold choice and error analysis.
Checkpoint: Defend why your selected model meets the actual objective.

WEEK 3 / LLMS + GROUNDED APPLICATIONS

Study topics 13-16. Build a small document Q&A; workflow with citations and permission-aware retrieval. Create answerable and unanswerable evaluation questions. Compare retrieval and generation failures. Explain when fine-tuning or tool use would add value.
Deliverable: A reproducible demo plus evaluation results and failure examples.
Checkpoint: Explain RAG vs fine-tuning and agent vs fixed workflow with tradeoffs.

WEEK 4 / SHIP + INTERVIEW

Study topics 17-20. Serve your model behind a validated API; containerize it, track artifacts and document rollout/rollback. Add basic data/service monitoring. Practice the five scenarios and two timed mock interviews.
Deliverable: A project README with architecture, setup steps, metrics, limitations and monitoring plan.
Checkpoint: Give a two-minute project walkthrough and a ten-minute system design answer.

ADJUST TO YOUR TARGET ROLE

ML Engineer: Go deeper on data, classical ML, evaluation, deployment and monitoring.
GenAI Engineer: Go deeper on LLMs, RAG evaluation, fine-tuning choices, tool reliability and serving.
Both tracks need Python, SQL, clear reasoning and evidence of working systems.

Practice, then make it yours

A 45-MINUTE MOCK INTERVIEW

5 min: project introduction and your contribution.
10 min: one Python or SQL task with edge cases.
10 min: explain two randomly selected topics.
10 min: debug a scenario using evidence.
10 min: design a small system with cost, latency and failure tradeoffs.

SELF-SCORE EACH ANSWER / 0-2 PER ITEM

Clarity: Did I answer directly and define key terms?
Correctness: Did I avoid misleading claims?
Application: Did I connect the concept to a concrete example?
Evaluation: Did I explain how to test or measure the approach?
Judgment: Did I name tradeoffs and failure modes?
0 = missing, 1 = partial, 2 = clear. Revisit the weakest dimension; this is a practice rubric, not a hiring cutoff.

BEFORE YOU APPLY

Can you reproduce your project, explain its dataset and split, defend the metric, show error analysis, describe deployment and monitoring, and discuss what you would improve? Tailor preparation to the job description and use only authentic experience.

TECHNICAL REFERENCES / KEEP LEARNING

[1] scikit-learn: Common pitfalls and recommended practices
https://scikit-learn.org/stable/common_pitfalls
[2] Hugging Face PEFT: Quantization
https://huggingface.co/docs/peft/en/developer_guides/quantization
[3] MLflow: Experiment tracking
https://mlflow.org/docs/latest/tracking

The sample answers are original study examples, not company question banks. References support specific technical details; library APIs evolve.

Want the roadmap + interview guide?

DM me “AI” to get the AI/ML job-ready roadmap and interview questions & answers guide. Save this guide and practice one answer every day.